If we want to create a custom analyzer with a configured `mapping` character filter, we follow the same process as earlier to create an index with analyzer settings and required filters. This example shows the procedure for customizing a `keyword` analyzer to attach a `mapping` character filter.

Listing 7.38 A `keyword` analyzer with a `mapping` character filter

```
PUT index_with_mapping_char_filter
{
  "settings": {
    "analysis": {
      "analyzer": {
        "my_social_abbreviations_analyzer": {
          "tokenizer": "keyword",
          "char_filter": [
            "my_social_abbreviations"
          ]
        }
      },
      "char_filter": {
        "my_social_abbreviations": {
          "type": "mapping",
          "mappings": [
            "LOL => laughing out loud",
            "BRB => be right back",
            "OMG => oh my god"
          ]
        }
      }
    }
  }
}
```

Custom analyzer with a mapping character filter

Declares a character filter

Expands the defined character filter with mappings

Specifies the name of the filter: mapping, in this case

Provides a set of mappings in the mappings object as name-value pairs

We've created an index with custom analyzer settings and provided mappings in the character filter. Now we can test it using the `_analyze` API.

Listing 7.39 Testing the custom analyzer

```
POST index_with_mapping_char_filter/_analyze
{
  "text": "LOL",
  "analyzer": "my_social_abbreviations_analyzer"
}
```

The text results in `"token" : "laughing out loud"`, which indicates that "LOL" was replaced with the full form, "laughing out loud".

7.6.3 Mappings via a file

We can also provide a file containing mappings rather than specifying them in the definition. Listing 7.40 uses a character filter with mappings loaded from an external